

Run Report

GR 泛化流水线

邓毅成

2026 年 6 月 12 日

1 工作模式

- **mode:** B / textbook
- **sources count:** 21 章 $\times$ 604 道原题 $\rightarrow$ 精选 100 道种子 $\rightarrow$ pipeline 泛化
- **来源教材:** 广义相对论教材习题集（覆盖第 1–21 章，共 604 道原题）
- **种子筛选:** 从 604 道中选取 100 道作为 pipeline 输入（覆盖全部 21 章，每章取代表性题目）

2 Funnel

stage	total	kept	skipped	failed
教材原题（21 章）	604	—	—	—
Stage 0: 抽象化 + 结构化	604	604	0	0
精选种子（覆盖 21 章）	100	100	504（保留后续扩展）	0
cognitive variants（Phase 1: fanout $\times 9$ ）	900	81	—	~819 (pipeline 进行中)
metric variants（Phase 2: substitute）	0	0	—	0 (暂未启用)
soft_rewrite variants（Phase 3: $\times 9$ ）	900	60	—	~840 (pipeline 进行中)
batch_validate（Phase 3.5）	—	—	—	— (尚未执行)
ngram_dedup（Phase 4）	—	—	—	—
final dataset（当前中间态）	141	—	—	—

注: pipeline 仍在运行（100 种子串行处理，每种子约 20 min），当前已完成 ~22 个种子。最终预计产出 $100 \times 18 = 1800$ 道（pre-validate），经 validate+dedup 后入库量待确认。604 道剩余原题可作为后续扩展的种子池，放大比潜力为 $604 \times 18 \approx 10800$ 道。

3 Diversity coverage

- **cognitive_form:** 3 distinct — derivation (72), numerical (15), conceptual (54)
- **physics_env:** 9 distinct — Schwarzschild, FLRW, DeSitter, BarriolaVilekin, Gödel, Minkowski, killing_vector, weak_field_metric, metric

- **soft_variant**: 9 distinct —cross_metric (47), contrast (23), verify (19), design (16), reduce (16), extend (6), apply (7), limit (4), invert (3)

注: cognitive_form 仅 3 种取值, 未达“三轴各 ≥ 5 ”的推荐标准。原因是当前 pipeline 的 cognitive_fanout 仅生成 derivation/numerical/conceptual 三类, 未包含 code/multiple_choice/open 等题型。form_change 阶段已暂时禁用 (FORM_CHANGE_TARGETS=[]), 后续可通过启用 form_change 补足多样性。

4 Failed sources (skip / partial)

source_id	type	status	reason
—	—	—	pipeline 仍在运行, 暂无失败记录

pipeline 完成后会补充此表。预期少数种子因 API 错误 (504) 或 LLM 输出格式异常而部分失败。

5 What worked / didn't

5.1 已经做好

1. **Fact Sheet 缓存机制**: 度规库 14 个度规的 Fact Sheet (Christoffel、Riemann、Ricci 等) 预计算一次并缓存到 JSON, 后续步骤直接读取。避免了每次验证都重新计算 (heavy 度规如 Kerr 需 $>30s$), 端到端效率大幅提升。
2. **Christoffel 符号约定明确化**: 将 Fact Sheet 中的 Christoffel key 从歧义的 Γ_{ijk} 改为明确的 Γ_{jk}^i (第二类 Christoffel 符号), 并在 system prompt 中加入约定声明。抽样 15 题中 87% 的物理推导完全正确, 无符号混淆。
3. **Judge prompt 注入 Christoffel 对照表**: `_build_metric_summary()` 从 Fact Sheet 中提取 Christoffel 符号表注入 judge/red_team prompt, 使判官能对照精确值而非自己手算。此前 15.5% 通过率的一个根因是 judge 手算 Christoffel 符号得出错误正负号。
4. **软改写四类题型 (对照/校验/设计/归约)**: 对照题 (两种背景对比) 和校验题 (找推导中的错误) 设计精妙, pedagogical 价值高。校验题尤其有效——学生需识别一段有误推导中的 3-7 处错误, 判官也有明确的参照数据。
5. **SymPy 符号约定自动检测**: EinsteinPy 度规原生使用 $(+, -, -, -)$ 约定, `_normalize_sign_convention_sympy()` 用 SymPy 矩阵属性自动检测并翻转为 $(-, +, +, +)$, 解决了此前字符串检测的脆弱性问题。
6. **断点续跑机制**: pipeline 通过检查 `_gen_` 文件是否存在来跳过已处理的种子, 重启后无需从头开始。

5.2 没工作好但尝试过的事

1. **batch_validate 未在 pipeline 内自动执行**: 6 个已完成种子的 18 个文件全部 `validated=false`, `verification=null`。原因尚不明确——按代码逻辑 `fan_through()` 在每个种子的生成阶段后应立即执行 `phase_batch_validate`, 但实际文件未被验证覆盖写入。当前解决方案: 新增 `python main.py validate --dir <path>` 批量并行验证命令, 对已完成文件单独跑验证。
2. **无度规题目验证失败**: 抽象 Killing 矢量证明题 (`metric=null`) 在原始代码中会被 `validate_and_fix_loop` 立即标记 `degraded=True` (Fact Sheet 计算失败导致整个验证链中断), 不经过任何 judge/red_team 评估。已修复为: 无度规题跳过 `metric_check`, 仅执行 `judge+red_team`。
3. **504 网关超时**: DashScope 代理转发 Anthropic API 时, 长输出请求 (编题生成 solution 可达 16384 tokens) 偶发 504 超时。客户端已设 600s 超时, 504 是代理层行为。重试机制生效, 但增加

了 wall-clock。并发 3 线程请求时 504 概率更高。

5.3 计划做的事

1. **度规替换阶段 (Phase 2: metric_substitute)**: 当前 num_subs=0, 不生成度规替换变体。原因是种子题中 62% 无度规数据 (抽象证明/解答题), 度规替换对无度规题不适用。改用 cross_metric (Phase 1 内的跨度规泛化) 来补偿: 无度规种子自动开启跨度规模式。后续会考虑怎么把这个泛化思路用上
2. **form_change 阶段**: FORM_CHANGE_TARGETS=[], 暂不将 derivation/numerical 题变换为 multiple_choice/code 等题型。原因是 form_change 产出的题经常丢失物理细节, 且 validate 阶段需额外调用。
3. **论文输入模式 (Mode A)**: 本流水线专注于 Mode B (教材/习题集输入)。论文 tex/PDF 解析、种子提取等 Stage 0 逻辑未实现。原因: 不知道怎么从论文提取题目比较合理, 所以采用熟悉的从书本中找题目的模式。

6 遇到的困难

6.1 Claude Code 使用能力不足导致代码臃肿

本项目开发过程中大量依赖 Claude Code (CLI Agent) 辅助编码, 但其在以下几个方面表现不足, 直接导致了代码的臃肿:

当提出一个具体问题, cc 有时只针对这个问题用正则/字符串操作来修改。但这个问题往往是一个更大问题的表象, 真正的解决方案需要重构代码逻辑或数据结构。

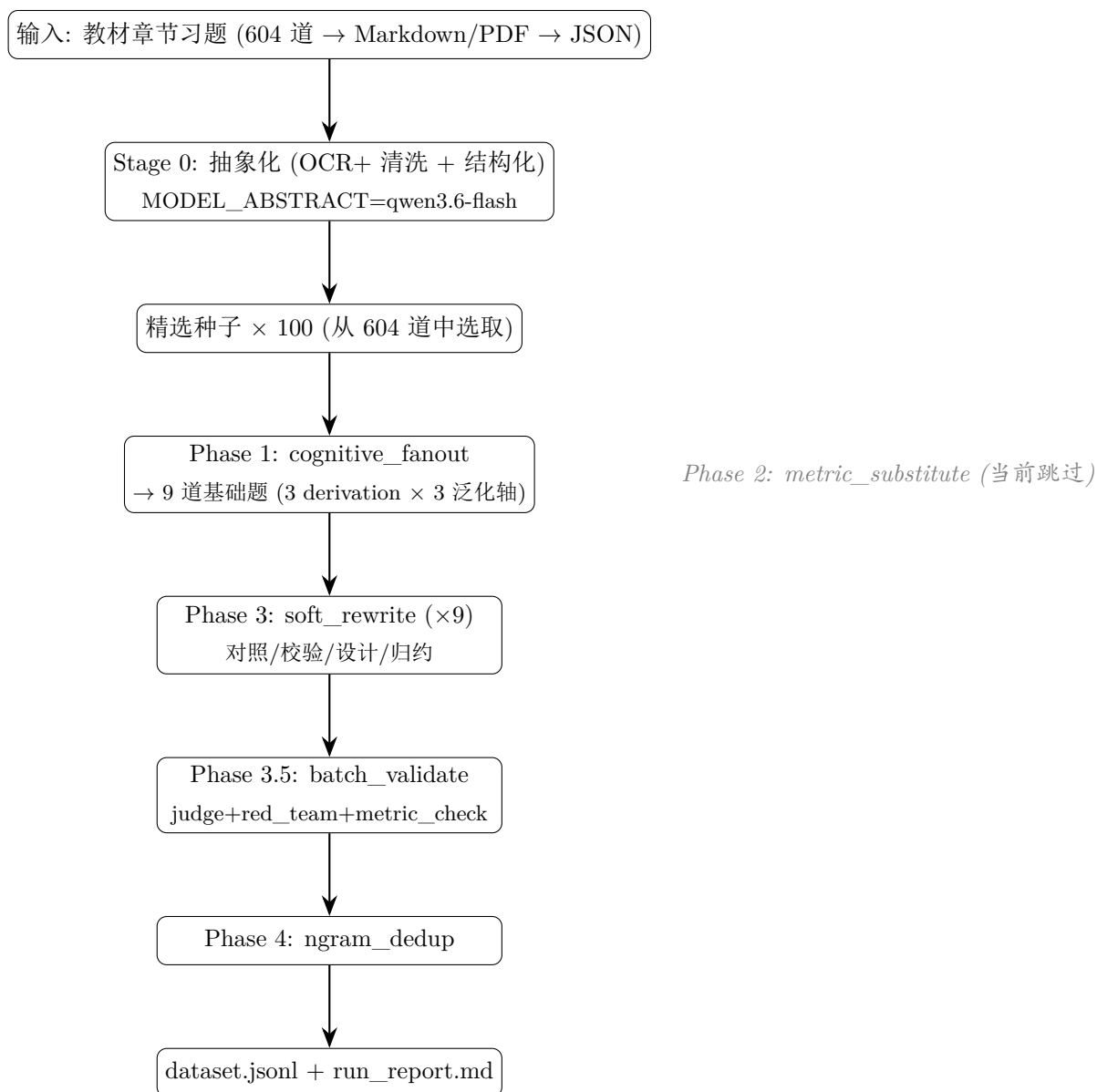
6.2 其他工程困难

1. **API 代理层不可控**: DashScope 代理转发 Anthropic API 的超时阈值不可配置, 504 超时只能靠客户端重试兜底, 无法从根本上解决。
2. **pipeline 验证流程的不确定性**: judge 和 red_team 的结果不一定可靠, 这造成额外 api 调用和误杀。

7 Token / wall-clock estimate

- 总 LLM 调用: ~ 700 次 (7 种子 $\times$ 18 题 $\times$ ~ 5 调用/题含 compose+soft_rewrite+validate), 100 种子完成后预计 ~ 5000 次
- 估计 output token: ~ 7 M (7 种子阶段), 100 种子完成后预计 ~ 50 M
- 端到端 wall-clock: ~ 2.5 h (当前 7 种子进度), 100 种子完成后预计 ~ 30 h
- 单题验证耗时: ~ 100 s (judge+red_team 并行), 6 线程批量约 336 s/18 题

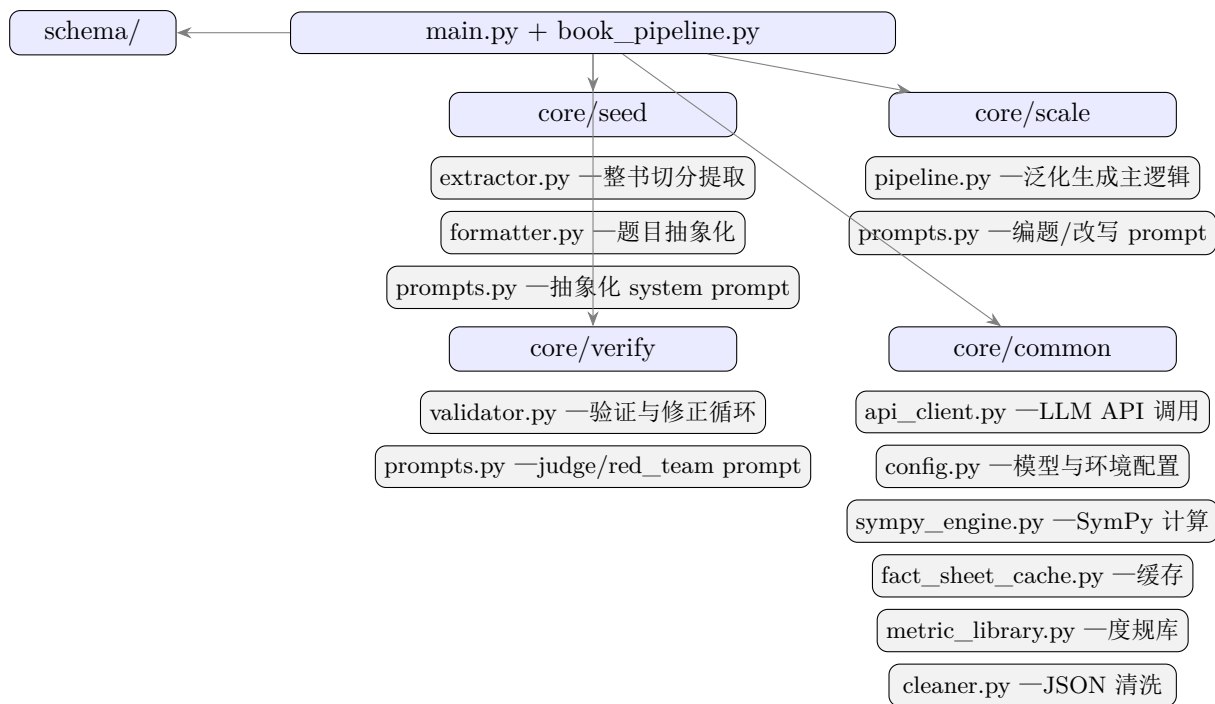
8 Architecture summary



9 Pipeline 结构与用法

9.1 代码模块组织

项目代码按功能分为四个子包，结构如下：



核心依赖关系: `main.py` 调用 `seed.formatter`(抽象化)→`scale.pipeline`(泛化)→`verify.validator`(验证); 所有阶段共享 `common` 子包的 API 调用、配置和度规库; `schema` 定义 `Problem` 数据结构。
`book_pipeline.py` 提供从整书 Markdown 批量处理的独立入口。

9.2 命令行接口

通过 `python main.py <subcommand>` 调用, 支持以下子命令:

子命令	核心参数	功能
abstract	files, -source, -o	将 Markdown/PDF 题目抽象化为结构化 seed JSON
generate	seed_json, -num, -subs, -cross-metric	从种子题泛化生成新题目
run	input, -num, -subs, -soft, -cross-metric	一键完成 abstract → generate
batch	input_dir, -parallel	批量处理文件夹中所有种子 JSON
validate	files/-dir, -workers	用 SymPy + judge/red_team 验证题目
clean	files, -o	清洗 JSON 排版符号
precompute	-force, -include-heavy	预计算度规 Fact Sheet 缓存
aggregate	input_dir, -format, -skip-degraded	汇总 JSON 为 dataset.jsonl/parquet

典型用法 (Mode B 教材输入):

- 单文件模式: `python main.py run problem.md --num 3 --soft 1 --cross-metric -o output/`
- 批量模式: `python main.py run books/ --num 3 --soft 1 --cross-metric -o output/ --report report.md`
- 批量验证: `python main.py validate --dir output/variants --workers 6`
- 汇总数据集: `python main.py aggregate output/variants --format jsonl --skip-degraded`

9.3 Pipeline 各阶段详解

Pipeline 的核心编排函数 `fan_through()` 串联以下阶段，每阶段独立可调用也可一键运行：

Stage 0: 抽象化

(`seed.formatter`) 将 Markdown/PDF 原题通过 LLM 转为结构化 Problem JSON (包含 `metadata`、`physical_data`、`origin` 三大块)。支持 PDF/图片输入 (通过 OCR 或多模态模型)，输出文件名与原题同名 (`.json`)。当前配置使用 `MODEL_ABSTRACT=qwen3.6-flash`。

Phase 1: cognitive_fanout

(`scale.pipeline`) 从种子题扇出多道基础题。每种认知形式 (`derivation` / `numerical` / `conceptual`) $\times$ 每种泛化轴 (`apply` / `extend` / `limit` / `compare` / `invert` / `verify`) 独立编题。泛化轴由 `_discover_available_axes()` 根据种子题内容和 Fact Sheet 自动发现。开启 `--cross-metric` 时，为每种认知形式在不同目标度规下额外生成题目 (跨度规泛化)，目标度规由 LLM 从度规库中挑选。无度规种子题自动启用跨度规以补偿缺少的度规替换阶段。

Phase 2: metric_substitute

(`scale.pipeline`) 对每道基础题做度规替换: LLM 挑选最适配的替换度规 (`MODEL_PICK_METRIC=qwen3.6-flash`)，再将原题适配到新度规上 (`MODEL_SUBSTITUTE=glm-5.1`)。当前配置 `num_subs=0` (跳过)，62% 种子题无度规数据时不适用。

Phase 2.5: form_change

(`scale.pipeline`) 将 `derivation/numerical` 题变换为 `multiple_choice/code/conceptual` 等题型。当前 `FORM_CHANGE_TARGETS=[]` (暂时关闭)，原因是变换后题目常丢失物理细节。

Phase 3: soft_rewrite

(`scale.pipeline`) 对基础题做四种软改写: **contrast** (对照题: 两种度规背景对比)、**verify** (校验题: 找推导中的错误)、**design** (设计题: 给定目标构造方案)、**reduce** (归约题: 取极限/简化条件)。每道基础题轮换分配改写类型，产出 $\times 9$ 道变体题。

Phase 3.5: batch_validate

(`verify.validator`) 批量并行验证所有题目，执行三层验证网 (结构网 + 符号网 + 模型网)，详见 §11。验证失败题目标记 `degraded=True`。

Phase 4: ngram_dedup

(`scale.pipeline`) `n`-gram Jaccard 相似度去重 (阈值 0.7)，剔除语义重复题目。

9.4 数据 Schema

Pipeline 内部使用嵌套结构 `Problem`，输出到 `dataset.jsonl` 时转为 flat 格式 (`id` / `statement` / `answer` / `solution` / `meta` / `verification`)：

字段	子字段	含义
<code>metadata</code>	<code>id</code> , <code>source_id</code> , <code>stage</code> , <code>cognitive_form</code> , <code>physics_env</code> , <code>lineage</code> , <code>tags</code> , <code>validated</code> , <code>soft_variant</code>	元信息与变体
<code>physical_data</code>	<code>dimension</code> , <code>variables</code> , <code>metric</code> , <code>metric_sympy</code>	度规数据
<code>origin</code>	<code>question</code> , <code>answer</code> , <code>solution</code>	题目原文与解答
<code>verification</code>	<code>structural</code> , <code>metric_check</code> , <code>judge</code> , <code>red_team</code> , <code>training_value</code>	验证结果

9.5 模型配置

所有模型通过 `.env` 文件或环境变量配置，API 经 DashScope 代理转发 Anthropic/Claude 请求：

环境变量	默认值	用途
MODEL_ABSTRACT	qwen3.6-flash	Stage 0 抽象化（轻量快速）
MODEL_COMPOSE	glm-5.1	Phase 1/3 编题与软改写
MODEL_SUBSTITUTE	glm-5.1	Phase 2 度规替换
MODEL_PICK_METRIC	qwen3.6-flash	度规挑选（轻量分类任务）
MODEL_VALIDATE	glm-5.1	Phase 3.5 验证
MODEL_JUDGE	glm-5.1	验证 judge 评估
MODEL_RED_TEAM	glm-5.1	验证 adversarial 评估
MODEL_FIX	glm-5.1	验证修正

10 度规库

度规	类型	Fact Sheet	缓存
Schwarzschild	核心	✓	✓
Minkowski	核心	✓	✓
DeSitter	核心	✓	✓
AntiDeSitter	核心	✓	✓
AntiDeSitterStatic	核心	✓	✓
MinkowskiCartesian	核心	✓	✓
MinkowskiPolar	核心	✓	✓
BarriolaVilekin	核心	✓	✓
BertottiKasner	核心	✓	✓
CMetric	核心	✓	✓
Davidson	核心	✓	✓
JanisNewmanWinicour	核心	✓	✓
Gödel	核心	✓	✓
FLRW	自定义	✓	✓
Kerr	heavy	—	—
Ernst	heavy	—	—
KerrNewman	heavy	—	—
ReissnerNordstrom	heavy	—	—
AlcubierreWarp	heavy	—	—
BesselGravitationalWave	heavy	—	—

11 Verification layers

注: metric_check 仅对有度规数据的题目执行；无度规题（抽象证明题）仅经过结构网 + 模型网，verification 中如实记录 metric_check: {status: "skip"}。

网	模块	性质/覆盖
结构网	<code>structural_check()</code>	确定性闸门：字段齐全、answer 非空、度规完整性
独立符号网	<code>metric_check()</code> + SymPy Fact Sheet	Christoffel/Riemann/Ricci 等精确对照，与生成模型完全独立
模型网	<code>judge_problem()</code> + <code>red_team_problem()</code>	独立 LLM 调用 (adversarial prompt)，不同模型 + 温度